

Numerisches Lösen von Nullstellen

SWP – Softwareentwicklung und
Projektmanagement

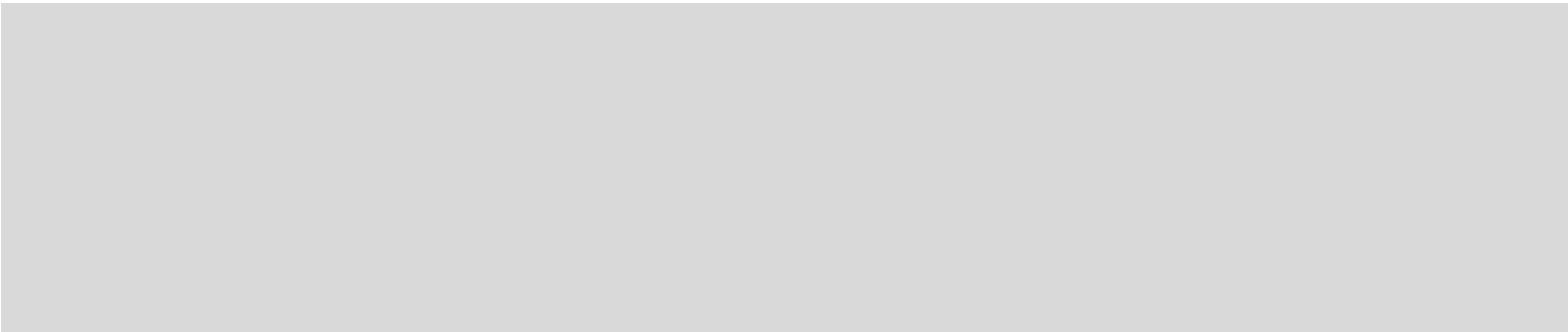


Schuljahr: 2025/26

Jahrgang: 3BHWII

Projektersteller: Akbulut Enes

Katalognummer: 01



Aufgabe 1**Grundlagen des Bisektionsverfahrens und Stetigkeit**

$$x^2 = 1 \quad | -1$$

$$x^2 - 1 = 0$$

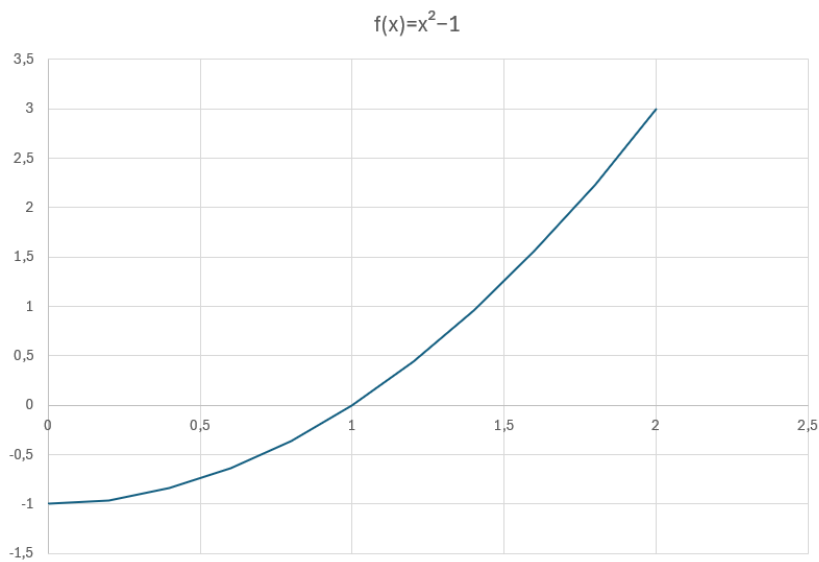
$$f(x) = x^2 - 1$$

$$[a, b] = [0, 2]$$

$$f(0) = 0^2 - 1 = -1$$

$$f(2) = 2^2 - 1 = 3$$

$$f(0) \cdot f(2) < 0$$



Die Nullstelle auf dem Graphen liegt bei $x = 1$. Stetig laut Definition heißt, dass man die Funktion, ohne den Stift abzusetzen, zeichnen kann. Wenn es keine Lücken, Unterbrechungen oder Sprünge hat, kann es auch als stetig bezeichnet werden. Die Funktion $f(x) = x^2 - 1$ ist stetig, da sie zur Gruppe der Polynomfunktionen gehören sie sind über ihrem gesamten Definitionsbereich (die reellen Zahlen) überall stetig.

Aufgabe 2**Funktionsweise der Intervallhalbierung (Bisektionsverfahren)**

Wenn $f(a) \cdot f(c) < 0$, dann b mit c ersetzen

Wenn $f(b) \cdot f(c) < 0$, dann a mit c ersetzen

Aufgabe 2) $[a, b] = [0; 2]$ Kataloglohn: $= 1$ $f(a) = -1$ $f(b) = 3$

Iteration 1) $c = \frac{0+2}{2} = 1$ $f(1) = 1^2 - 1 = 0$

Nullstelle schon gefunden.

Iteration 2) neues Intervall: $[0; 1]$ $f(a) = -1$ $f(b) = 0$

$c_2 = \frac{0+1}{2} = 0,5$ $f(c_2) = 0,5^2 - 1 = -0,75$

Iteration 3) neues Intervall: $[0,5; 1]$ $f(a) = -0,75$ $f(b) = 0$

$c_3 = \frac{0,5+1}{2} = 0,75$ $f(c_3) = 0,75^2 - 1 = -0,4375$

Iteration 4) neues Intervall: $[0,75; 1]$ $f(a) = -0,4375$ $f(b) = 0$

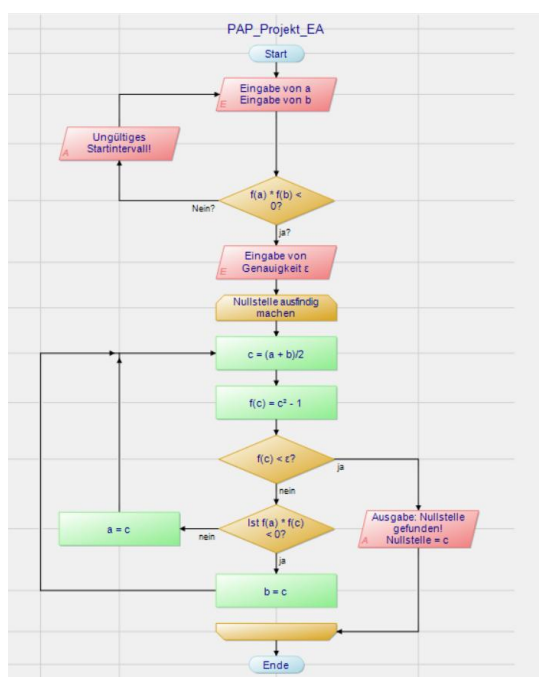
$c_4 = \frac{0,75+1}{2} = 0,875$ $f(c_4) = 0,875^2 - 1 = -0,234375$

Iteration 5) neues Intervall: $[0,875; 1]$ $f(a) = -0,234375$ $f(b) = 0$

$c_5 = \frac{0,875+1}{2} = 0,9375$ $f(c_5) = 0,9375^2 - 1 = -0,1211$

das nächste Intervall: $[0,9375; 1]$

Die Nullstelle wurde nach der 1. Iteration schon gefunden. Wenn man aber weiter iteriert, dann merkt man, dass sich a immer weiter zur Nullstelle nähert, aber es nie erreicht.

Aufgabe 3**Programmablaufplan (PAP) des Bisektionsverfahrens**

Aufgabe 4

Projektplanung und Vorgehensmodell

Vorgehensmodell

Für dieses Projekt habe ich das **Wasserfallmodell** gewählt.

Das Wasserfallmodell passt gut, weil das Projekt klar aufgebaut ist und die Schritte nacheinander durchgeführt werden können. Zuerst wird geplant, dann programmiert, danach getestet und am Ende dokumentiert.

Da es sich um ein überschaubares Schulprojekt handelt, ist dieses Modell einfach anzuwenden und gut geeignet.

Programmierparadigma

Für dieses Projekt habe ich das **strukturierte Programmieren** gewählt.

Ich habe das gewählt, da wir einen einfachen Code haben, der nur mittels den Bisektionsverfahren Nullstellen in einem Intervall findet. Es ist meiner Meinung nach nicht wert, es komplexer zu gestalten.

Zeitmanagement (geschätzte Dauer)

Für die Umsetzung des Projekts wurde folgende Zeit eingeplant:

Projektplanung: 3h

Programmstruktur planen: 2h

Programmierung: 5h

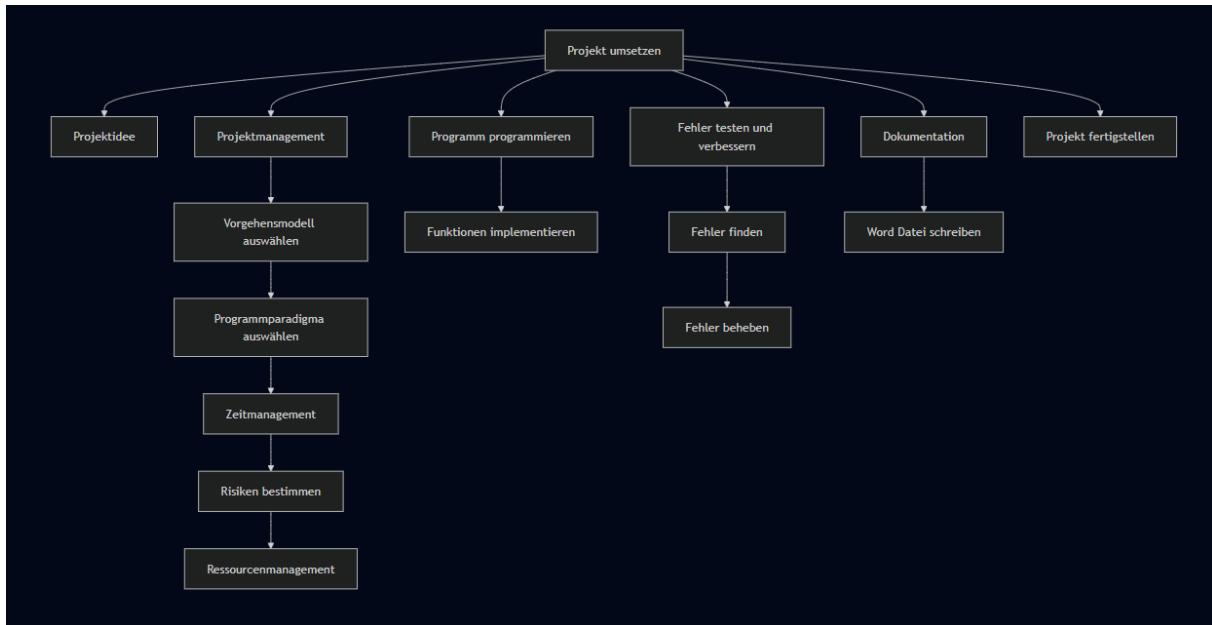
Testen: 3h

Dokumentation: 2,5h

Insgesamte Zeit: 15,5h

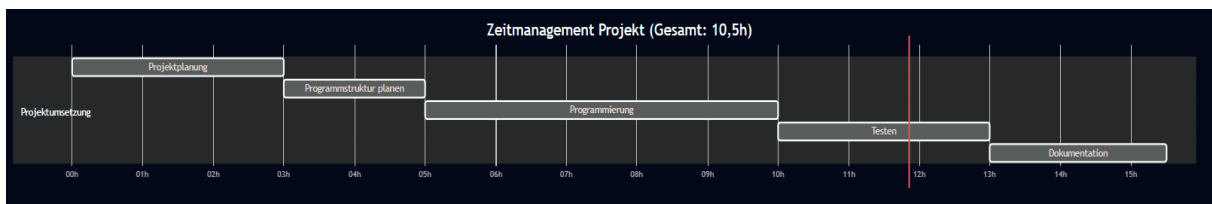
Work Breakdown Structure (WBS / PSP)

Das Projekt wurde in folgende Arbeitsschritte unterteilt:



Diese Schritte helfen dabei, das Projekt übersichtlich zu planen.

Gantt-Diagramm



Dabei wurde regelmäßig kontrolliert, ob der Fortschritt dem Plan entspricht.

Ressourcenmanagement

Für das Projekt wurden folgende Ressourcen benötigt:

Personal

- 1 Entwickler (Schüler)

Hardware

- Laptop oder PC

Software

- Python
- Entwicklungsumgebung (z. B. VS-Code)

- Zusatzbibliotheken (falls benötigt)
- Word für Dokumentation

Diese Ressourcen sind ausreichend für die Umsetzung des Projekts.

Risikomanagement

Mögliche Risiken wurden in **innere Risiken** und **äußere Risiken** unterteilt.

Innere Risiken

- Programm funktioniert nicht richtig
- Zeitplanung wird nicht eingehalten
- Fehler im Code

Äußere Risiken

- Computer funktioniert nicht
- Softwareprobleme
- Krankheit vor der Abgabe

Veröffentlichungsprüfung (Lizenz & Urheberrecht)

Vor der Veröffentlichung der Software wird überprüft, ob verwendete Programme und Bibliotheken frei verwendet werden dürfen.

Python steht unter der **PSF-Lizenz** und darf kostenlos verwendet werden.

Auch verwendete Bibliotheken wurden auf ihre Lizenz geprüft.

Es wurden keine geschützten Inhalte ohne Erlaubnis verwendet. Deshalb darf die Software veröffentlicht werden.

Aufgabe 5

Implementierung des Bisektionsverfahrens in Python

```

1  def f(x: float) -> float:
2      """
3      Diese Funktion wertet die eingegebene Funktion an der Stelle x aus.
4      """
5      return eval(func_input)
6
7
8  func_input = input("Funktion eingeben (z.B. x**2 - 1): ")      #Funktion eingeben
9
10 a = float(input("a eingeben: "))                                #Anfangsintervall eingeben
11 b = float(input("b eingeben: "))                                #Endintervall eingeben
12
13
14 # Prüfen ob Intervall gültig ist
15 if f(a) * f(b) >= 0:
16     print("Ungültiges Intervall!")
17     raise SystemExit()
18
19
20 exponent = int(input("Genauigkeit eingeben (Exponent für 10^x): ")) #Exponent eingeben
21 epsilon = 10 ** exponent                                           #Genauigkeit berechnen
22
23 while True:
24
25     c = (a + b) / 2
26     fc = f(c)
27
28     # richtige Abbruchbedingung
29     if abs(fc) < epsilon:
30         print("Nullstelle gefunden:", c)                            #Nullstelle gefunden
31         print(f"Endintervall: [{a}, {b}]")                          #Endintervall ausgeben
32         break
33
34     if f(a) * fc < 0:
35         b = c                                                        #wenn funktion von a und von c zusammen multipliziert negativ ist,
36     else:                                                            #dann wird b zu c, sonst wird a zu c
37         a = c

```

```

Funktion eingeben (z.B. x**2 - 1): x**2 - 25
a eingeben: 0
b eingeben: 100
Genauigkeit eingeben (Exponent für 10^x): -15
Nullstelle gefunden: 5.0
Endintervall: [4.999999999999998, 5.000000000000002]

```

```

Funktion eingeben (z.B. x**2 - 1): x**2 - 81
a eingeben: 0
b eingeben: 100
Genauigkeit eingeben (Exponent für 10^x): -15
Nullstelle gefunden: 9.0
Endintervall: [8.999999999999996, 9.000000000000002]

```

```

Funktion eingeben (z.B. x**2 - 1): x**2 - 144
a eingeben: 0
b eingeben: 100
Genauigkeit eingeben (Exponent für 10^x): -15
Nullstelle gefunden: 12.0
Endintervall: [11.999999999999999, 12.000000000000001]

```

Der oben dargestellte Code implementiert einen Solver zur Berechnung von Nullstellen mit der Methode der Intervallhalbierung (Bisektion). Die Funktionsfähigkeit des Programms wurde mit den Testfunktionen $x^2 - 25$, $x^2 - 81$ und $x^2 - 144$ überprüft. Die berechneten Nullstellen stimmen mit den erwarteten Ergebnissen (5, 9 und 12) überein, wodurch bestätigt wird, dass der Solver korrekt funktioniert.

Aufgabe 6

Implementierung des Regula-Falsi-Verfahrens

```

1  def f(x: float) -> float:
2      """
3      Diese Funktion wertet die eingegebene Funktion an der Stelle x aus.
4      """
5      return eval(func_input)
6
7
8  func_input = input("Funktion eingeben (z.B. x**2 - 1): ")      #Funktion eingeben
9
10 a = float(input("a eingeben: "))                                #Anfangsintervall eingeben
11 b = float(input("b eingeben: "))                                #Endintervall eingeben
12
13
14 # Prüfen ob Intervall gültig ist
15 if f(a) * f(b) >= 0:
16     print("Ungültiges Intervall!")
17     raise SystemExit()
18
19
20 exponent = int(input("Genauigkeit eingeben (Exponent für 10^x): ")) #Exponent eingeben
21 epsilon = 10 ** exponent                                           #Genauigkeit berechnen
22
23 while True:
24
25     # Funktionswerte berechnen
26     fa = f(a, func_input)
27     fb = f(b, func_input)
28
29     # Regula-falsi-Formel zur Berechnung der neuen Näherung
30     c = b - fb * (b - a) / (fb - fa)
31
32     # Funktionswert an neuer Stelle berechnen
33     fc = f(c, func_input)
34
35     # richtige Abbruchbedingung
36     if abs(fc) < epsilon:
37         print("Nullstelle gefunden:", c)          #Nullstelle gefunden
38         print(f"Endintervall: [{a}, {b}]")        #Endintervall ausgeben
39         break
40
41     if fa * fc < 0:
42         b = c                                     #wenn funktion von a und von c zusammen multipliziert negativ ist,
43     else:                                         #dann wird b zu c, sonst wird a zu c
44         a = c

```

```

=== Nullstellen-Solver mit Regula falsi ===
Funktion eingeben (z.B. x**2 - 25): x**2 - 25
a eingeben: 0
b eingeben: 100
Exponent für Genauigkeit eingeben (z.B.10**-4): -10
Nullstelle gefunden: 4.999999999999999
Endintervall: [4.999999999999999, 100.0]

```

```

=== Nullstellen-Solver mit Regula falsi ===
Funktion eingeben (z.B. x**2 - 25): x**2 - 81
a eingeben: 0
b eingeben: 100
Exponent für Genauigkeit eingeben (z.B.10**-4): -10
Nullstelle gefunden: 8.999999999999999
Endintervall: [8.999999999999999, 100.0]

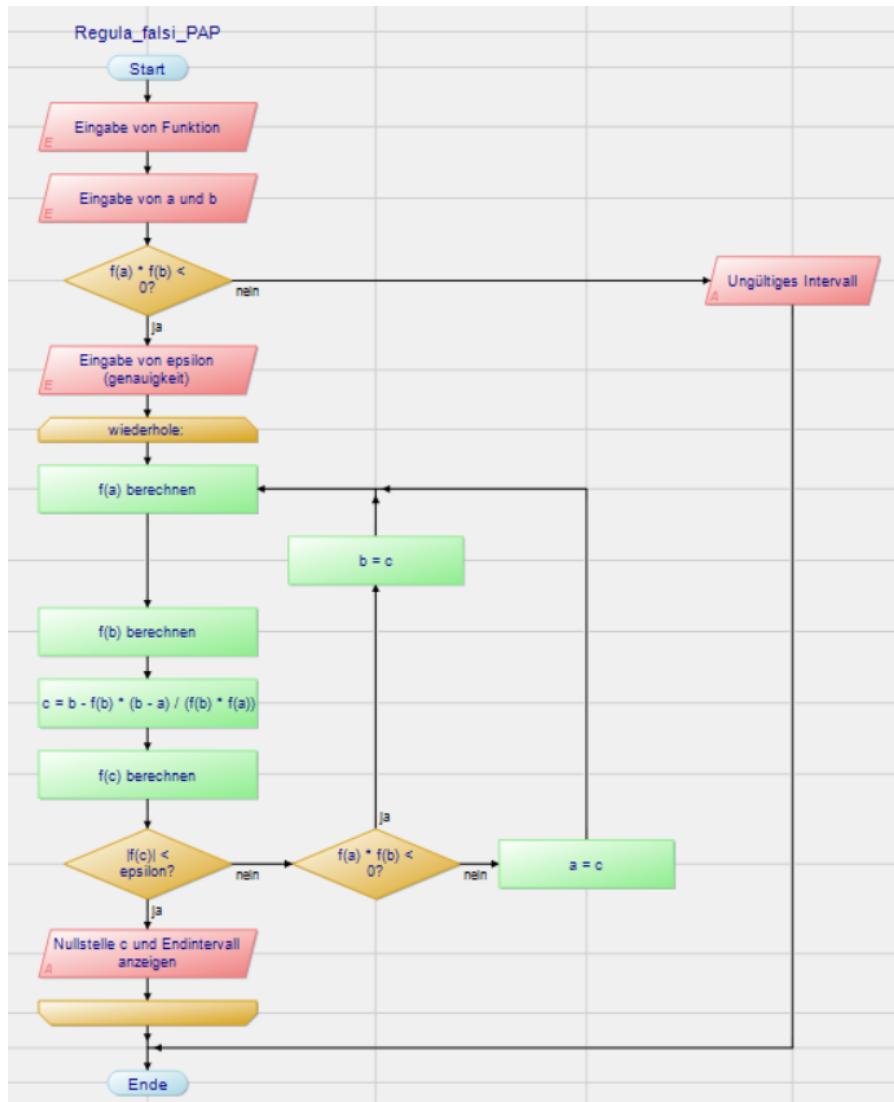
```

```

=== Nullstellen-Solver mit Regula falsi ===
Funktion eingeben (z.B. x**2 - 25): x**2 - 144
a eingeben: 0
b eingeben: 100
Exponent für Genauigkeit eingeben (z.B.10**-4): -10
Nullstelle gefunden: 11.999999999999999
Endintervall: [11.999999999999999, 100.0]

```

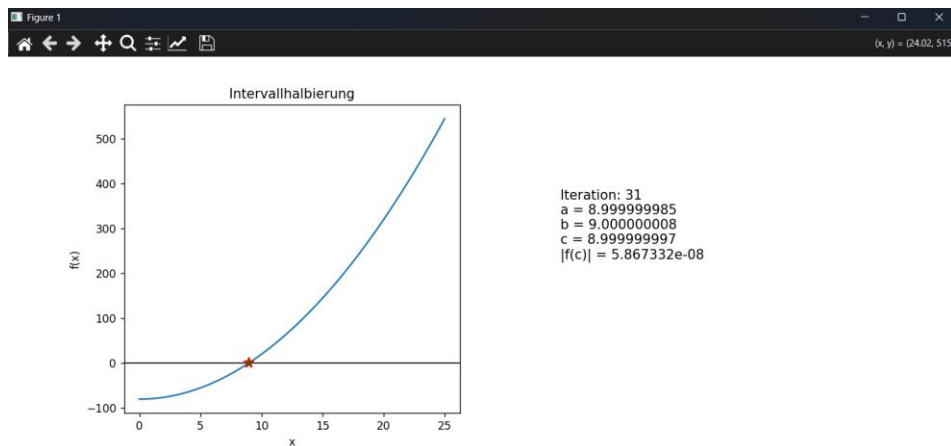
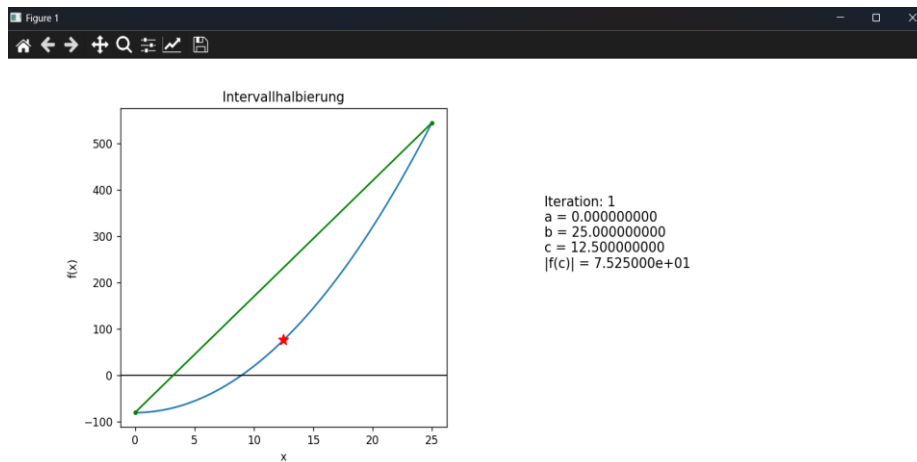
Bei der Methode der Regula Falsi wird das Intervall nicht immer symmetrisch verkleinert wie bei der Intervallhalbierung. Stattdessen nähert sich eine Intervallgrenze schneller der Nullstelle an. Daher bleibt in diesem Beispiel die obere Grenze b unverändert, während die Nullstelle trotzdem korrekt bestimmt wird.



Das ist die PAP für den Solver mittels Regula Falsi.

Aufgabe 7**Grafische Darstellung der Intervallhalbierung**

(Code in Github)

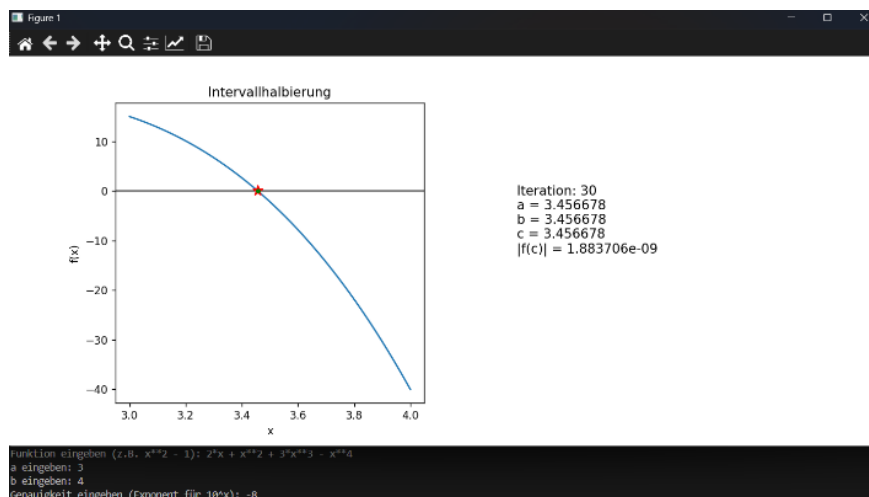
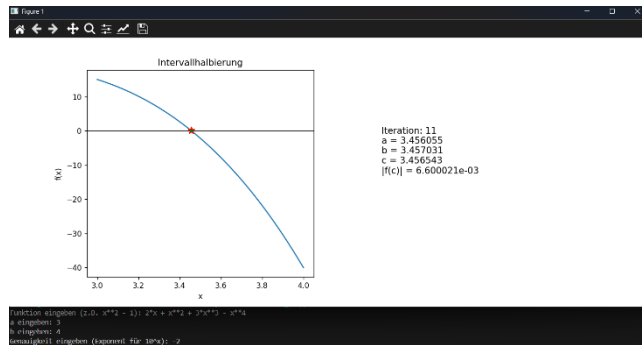


Die Animation zeigt die Intervallhalbierung zur Bestimmung einer Nullstelle einer Funktion. In jedem Iterationsschritt wird das Intervall $[a, b]$ verkleinert und der Mittelpunkt c als aktueller Näherungswert dargestellt. Zusätzlich werden rechts die aktuellen Werte von a , b , c sowie die Genauigkeit $|f(c)|$ angezeigt, wodurch die Annäherung an die Nullstelle sichtbar wird.

Aufgabe 8

Analyse der Iterationsanzahl bei unterschiedlicher Genauigkeit

Bei einer Genauigkeit von $\varepsilon = 10^{-2}$ konnte die Nullstelle des Polynoms nach 11 Iterationen näherungsweise bestimmt werden. Das Ergebnis liegt bei $x \approx 3,4565$ und stimmt bereits gut mit der erwarteten Lösung überein. Da die Genauigkeit geringer gewählt wurde, waren weniger Iterationsschritte notwendig.



Bei einer höheren Genauigkeit von $\varepsilon = 10^{-8}$ benötigte der Solver 30 Iterationen, um eine genauere Näherung der Nullstelle zu berechnen. Das Ergebnis liegt bei $x \approx 3,456678$ und stimmt sehr genau mit der erwarteten Lösung überein. Dies zeigt, dass mit steigender Genauigkeit auch die Anzahl der Iterationsschritte zunimmt.

Aufgabe 9

Berechnung der Leitungslänge mit numerischer Nullstellenbestimmung

```

1  import math # für sinh() und cosh()
2
3  def solve_aufg9():
4      def f(x: float) -> float:
5          return eval(func_input) # wertet eingegebene Funktion an Stelle x aus
6
7      func_input = input("Funktion eingeben (z.B. x**2 - 1): ") # Funktion eingeben
8      a = float(input("a eingeben: ")) # Startintervall
9      b = float(input("b eingeben: ")) # Endintervall
10
11     if f(a) * f(b) >= 0: # prüft Vorzeichenwechsel im Intervall
12         print("Ungültiges Intervall!")
13         raise SystemExit()
14
15     exponent = int(input("Genauigkeit eingeben (Exponent für 10^x): ")) # Genauigkeit festlegen
16     epsilon = 10 ** exponent # berechnet epsilon
17
18     while True:
19         c = (a + b) / 2 # Intervallmitte berechnen (Bisektion)
20         fc = f(c) # Funktionswert an Stelle c
21
22         if abs(fc) < epsilon: # Abbruchbedingung erreicht
23
24             print("Krümmungsradius:", c) # gefundene Nullstelle = Radius
25             print(f"Endintervall: [{a}, {b}]")
26
27             a = c # speichert Radius für Längenberechnung
28
29             l = 2 * a * math.sinh(100 / (2 * a)) # Seillänge berechnen
30
31             print("Die länge der Leitung ist:", l)
32             break
33
34         if f(a) * fc < 0: # Nullstelle liegt im linken Teilintervall
35             b = c
36         else:
37             a = c # sonst im rechten Teilintervall
38
39     # x*math.cosh(50/x) - x - 10 #Formel
40
41     if __name__ == "__main__":
42         solve_aufg9()
43

```

```

Funktion eingeben (z.B. x**2 - 1): x*math.cosh(50/x) - x - 10
a eingeben: 10
b eingeben: 200
Genauigkeit eingeben (Exponent für 10^x): -6
Krümmungsradius: 126.63242816925049
Endintervall: [126.63233757019043, 126.63251876831055]
Die länge der Leitung ist: 102.61868714093724
PS C:\Users\akbul\OneDrive - tgm - Die Schule der Technik\TGM\3BHWII\SWP\Projekt>

```

Negative Werte für den Krümmungsradius a sind nicht sinnvoll, da ein Radius physikalisch nicht negativ sein kann. Außerdem enthält die Funktion den Ausdruck $\cosh(50/a)$, weshalb $a = 0$ nicht erlaubt ist (Division durch 0). Daher wurde das Startintervall auf positive Werte eingeschränkt, z. B. $[10, 200]$.